SentinelMesh — Project Deep-Dive

Oct 1, 2026 · @GuluGulu

SentinelMesh is an AI-native cybersecurity threat-detection and attack-intelligence platform built as a phase-gated, 20-service microservice system with a non-fabrication engineering discipline: every claim in this document is backed by code, a real measured number, or an honestly-recorded gap — nothing here is invented for effect.

Problem & motivation

Traditional SOC tooling is built around three separate, poorly-connected layers: a SIEM that collects logs and runs static correlation rules, a rule-based IDS/IPS that flags known signatures, and an analyst who manually pivots between tools to reconstruct "what actually happened." This has three structural weaknesses:

1. **Rule-only detection misses novel behavior.** Signature and threshold rules catch what was already seen before; they do not generalize to a new technique that doesn't match a known pattern.
2. **Alerts are disconnected from the story.** A SIEM produces thousands of discrete alerts; it does not reconstruct the attack chain (initial access → lateral movement → exfiltration) an analyst actually needs to understand impact and respond.
3. **Analysts can't query data in their own language.** Hunting requires hand-written Cypher/SQL/KQL; most analysts either don't know the query language, or an LLM asked to help can hallucinate a wrong or dangerous raw query.

The problem statement this project set out to answer: can a single platform ingest raw security telemetry, detect anomalies *without relying only on static rules*, reconstruct the resulting events into an attack graph and attack chain automatically, explain that chain in natural language grounded only in real evidence, let an analyst hunt in plain English safely, and do all of this with the engineering discipline (tenancy, auth, observability, non-fabrication) a real production security product needs — not a prototype notebook.

The secondary motivation was academic/demonstrative: build a full, realistic, phase-gated enterprise system end-to-end — auth, event-driven microservices, a graph database, ML, LLM integration, Kubernetes — showing the complete lifecycle a real cybersecurity-AI product goes through, with every claim backed by a real measured result instead of a demo-only facade.

Solution & architecture

SentinelMesh ingests security telemetry (network flows, auth logs, DNS, process/endpoint, file-access events), normalizes it into a canonical event stream, builds a dynamic attack graph, runs anomaly + graph detection, reconstructs attack chains, maps them to MITRE ATT&CK, scores threats, and drives an analyst SOC UI with LLM-assisted explanation, natural-language hunting, safe attack simulation, deception, threat memory/prediction, attack-story reporting, and a production Kubernetes deployment.

It was built **phase by phase (19 phases, 0–18)** under a written "Engineering Constitution" — a self-imposed rulebook banning fabricated benchmarks, requiring real infrastructure in integration tests, server-side tenant isolation, and an honest `NOT VERIFIED` state wherever something genuinely wasn't tested, rather than a claimed one.

Pipeline, end to end:

1. `ingestion-gateway` — authenticated per-sensor telemetry intake (credential = `sensor_id.secret`, a different trust boundary from session auth), dedup, batch limits → Kafka `telemetry.raw`
2. `normalization-engine` — raw → canonical events on `events.canonical`; malformed messages to a dead-letter queue; optional threat-intel enrichment
3. `stream-processor` — canonical events → idempotent graph commands on `graph.commands`
4. `graph-service` — the only writer into Neo4j; applies each command as a parameterized, idempotent MERGE; serves the graph-query API and the threat-hunting Cypher compiler
5. `detection-engine` — adaptive anomaly detection (Isolation Forest) + stateful rule detectors over canonical events
6. `correlation-engine` — groups related detections into an attack chain (`forming → active`) and computes a threat score
7. `mitre-service` — maps chains/detections to the real, imported MITRE ATT&CK technique catalog
8. `ai-analyst` — grounded LLM explanations and a natural-language hunt planner (never raw Cypher — see below)
9. `reporting-service` — attack-storytelling reports that explicitly label every sentence as evidence / inference / prediction / synthetic
10. `api-gateway` — the BFF: local + OIDC auth, sessions, CSRF, deny-by-default RBAC, tenant scoping

11. `frontend/web` (Next.js) — the analyst SOC UI: dashboard, alerts, incidents, attack chains, MITRE heatmap, entity/attack-graph explorer, threat hunting, simulation, deception, threat memory, reports, benchmarks

Supporting data plane: PostgreSQL (control plane + pgvector for similarity search), Neo4j (the attack graph), Kafka/Redpanda (the durable event bus), Redis (cache/session/rate-limit, never the bus), MinIO (S3-compatible report/dataset storage), Keycloak (OIDC).

Why microservices, not a monolith: each stage of the pipeline (ingest → normalize → stream → graph → detect → correlate → explain) has a different scaling profile and failure mode — ingestion is I/O-bound and bursty, ML inference is CPU-bound, graph writes must be strictly serialized per-tenant. Splitting them lets each scale and fail independently, and enforces a hard rule: `graph-service` is the *only* writer to Neo4j, so graph consistency never has to be reasoned about across nine different callers.

Why event-driven (Kafka) over direct calls: telemetry arrives in bursts far faster than downstream processing (graph writes, ML inference) can keep up; a durable log absorbs the burst, gives at-least-once delivery with replay, and lets each consumer scale independently of the producer.

AI / ML components — what's actually used, and why

[Superseded by the Post-remediation update section below — read that one for the current, correct answer.] This paragraph is kept to show what the honest state was *before* that pass: at that point, the production, verified, running anomaly detector was classical machine learning (Isolation Forest) only. Two deep-learning architectures (an autoencoder and a graph neural network) were fully designed and wired into the serving code, but had no trained checkpoint yet — stated honestly rather than glossed over, per the project's rule to never claim a number that wasn't actually measured. **As of the remediation pass, both are now trained, with real checkpoints and real benchmark numbers** — see Post-remediation update.

Component	Category	Status	Why this technique
Isolation Forest (`sklearn.ensemble.IsolationForest`)	Classical ML, unsupervised ensemble of random trees	**Trained & deployed** — real model artifact serving live detections	Needs no labeled attacks (most real traffic is unlabeled); isolates anomalies by how *few* random splits separate them from the rest, which is cheap, robust to high-dimensional tabular network-flow features, and doesn't assume a Gaussian feature distribution the way a z-score baseline does
Statistical baseline (per-feature MAD z-score)	Classical statistics	**Implemented & benchmarked** — the platform's always-available fallback detector	Zero dependencies, zero training cost, interpretable per-feature — the floor every other method is measured against
Autoencoder (`Linear → ReLU` encoder/decoder, bottleneck architecture)	Deep learning (unsupervised reconstruction)	**Trained & deployed (updated)** — real checkpoint exists (`ml/artifacts/autoencoder_network_flow/v1/model.pt`); `ModelNotTrained` no longer applies. Real NSL-KDD benchmark: ROC-AUC 0.434 (worse than the other two methods — reported as measured)	Justified for higher-dimensional feature groups (network-flow, process-exec) where one reconstruction-error number captures *correlated* deviations across features that a per-feature z-score misses

Component	Category	Status	Why this technique
GNN: GraphSAGE / GAT (PyTorch + PyTorch Geometric)	Deep learning (graph representation learning)	**Trained & checkpointed (updated)** — real checkpoints exist for both GraphSAGE and GAT; the inference code now loads real trained weights, not just an architecture. Real held-out ROC-AUC ≈ 0.49 (near-random) — root cause: the test graph's forced topology carries no discriminative signal, not a training bug	The attack graph is relational, not tabular — a GNN can learn an entity's anomaly score from its *neighborhood* (who it talks to, what it accessed), which a per-row tabular model structurally cannot see. GraphSAGE (inductive, neighbor-sampling) and GAT (attention-weighted neighbors) were chosen as the two standard graph-anomaly architectures
LLM integration (provider-adapter pattern: `HttpLlmBoundary` in an Anthropic Messages-shaped HTTP boundary, or a network-free deterministic adapter for tests/no-credentials)	Pretrained Transformer (external API call, not trained by this project)	**Implemented, degrades honestly to "unsupported" with no API key — never a guessed answer**	Used for two things only: (1) explaining an already-detected, already-evidenced chain in natural language, and (2) translating a natural-language hunt question into a **closed, validated `QueryPlan` schema** — never a raw Cypher query the LLM could hallucinate or inject
Graph algorithms (clustering, structural anomaly heuristics, pgvector cosine-similarity search for threat memory)	Classical graph theory / vector search	**Implemented & running**	Always-available, no training step, fast baseline that degrades gracefully when the GNN path is unavailable

[This paragraph described the pre-remediation state; both are now trained — see the update below for what happened next.] At that point, training a GNN or autoencoder needed labeled, graph-structured training data and compute this project didn't yet have in scope; shipping a fabricated accuracy number for an untrained model would have violated the project's own non-fabrication rule. So the *serving infrastructure* for both deep architectures was built for real first (real forward pass, real checkpoint loading, real graceful degradation), specifically so training them later would be a data/compute problem, not a re-engineering one. That's exactly what happened: both were trained in the remediation pass, using that same serving infrastructure unchanged — real checkpoints, real benchmark numbers, including the unflattering ones.

Datasets

NSL-KDD (`KDDTrain+.txt`, 125,973 rows; `KDDTest+.txt`, 22,544 rows, 12,833 of them attacks) was the dataset used for both training and benchmarking.

Why NSL-KDD and not raw KDD Cup 1999: NSL-KDD is a cleaned, re-released version of the original KDD'99 intrusion-detection dataset that removes the duplicate/redundant records KDD'99 was criticized for (which biased models toward whatever attack type was most duplicated), and rebalances the difficulty across train/test splits. It remains one of the most widely used public labeled benchmarks for network intrusion detection, which matters for comparability — a result on NSL-KDD can be checked against decades of published IDS literature.

Why this dataset specifically, over CICIDS2017 / UNSW-NB15 / LANL (all considered): those three were investigated and rejected for this phase for a concrete, honest reason — CICIDS2017 was never actually downloaded (only an `.md5` checksum stub existed locally), UNSW-NB15 was only available as raw unlabeled partial packet captures, and LANL had no paired ground-truth redteam file staged locally. Rather than fabricate a benchmark number against data that wasn't actually there, the platform shipped adapters and results only for the one dataset that was genuinely present in usable, labeled form, and recorded the other three as **NOT VERIFIED — REQUIRES an actual labeled copy**, with their adapters deferred, not silently skipped.

What each row contains: 41 features per network connection — `duration`, `protocol_type`, `service`, `flag`, `src_bytes`, `dst_bytes`, plus content/traffic/host-based statistical features — and a label (`normal` or one of ~22 attack types grouped into DoS / Probe / R2L / U2R).

How it was used, end to end:

1. **Training** — `scripts/train_isolation_forest_network_flow.py` fits `sklearn.IsolationForest` on the full 125,973-row training split, reusing the exact same `sm_ml.features.extract_features` function the live detection pipeline uses at serving time (train/serve parity — a classic source of silent production bugs if skipped). The anomaly threshold is set at the 95th percentile of training scores. The artifact is versioned on disk (`ml/artifacts/isolation_forest_network_flow/v1/{model.joblib, metadata.json}`) and loaded read-only by `ml-inference`.
2. **Benchmarking** — a separate, independent harness (`sm_ml_training.benchmark`) re-implements ROC-AUC/precision/recall/F1/FPR from scratch in pure Python (deliberately no numpy/sklearn), so the metric computation itself is auditable rather than trusting a library as a black box, and runs both the statistical baseline and Isolation Forest against the held-out test split (see Benchmark results below).
3. **Real ingestion replay** — `scripts/ingest_nsl_kdd.py` replays real NSL-KDD rows through the actual authenticated `POST /api/v1/ingest/batch` endpoint (not a shortcut DB insert), mapping dataset columns to the platform's real `NetworkFlowPayload` schema, so the full pipeline (ingestion → normalization → graph → detection → correlation) is exercised against real data, not only synthetic simulation events.
4. **Web upload** — the SOC UI itself has a Datasets page (`/datasets`) that lets a user upload `KDDTrain+.txt` / `KDDTest+.txt` directly through the browser, which JIT-provisions a per-tenant sensor credential and replays the file through the identical real ingestion path — this is what a professor/interviewer can be shown live.

Every event produced by dataset replay is real, unlabeled-at-ingestion telemetry, as distinct from `simulation-service`'s synthetic attack scenarios, which are explicitly tagged `simulated: true` end-to-end and visibly badged "SIMULATION" in the UI — the platform never lets a synthetic event be mistaken for a real one.

Benchmark results (real, executed runs — not projected)

Both methods were fit on the same benign-only subset of the 125,973-row train split (modeling "normal" behavior — exactly how `detection-engine` uses the statistical model in production) and scored one row at a time against the identical 22,544-row test split (12,833 of them real attacks), matching the platform's real one-event-at-a-time serving contract rather than an unrealistic batch score production never actually takes.

Metric	Statistical baseline (MAD z-score)	Isolation Forest
ROC-AUC	0.639	**0.935**

Metric	Statistical baseline (MAD z-score)	Isolation Forest
Precision	0.581	**0.961**
Recall	**0.682**	0.621
F1	0.628	**0.755**
False-positive rate	0.649	**0.033**
Mean latency / row	**0.054 ms**	24.8 ms

Reading this honestly: Isolation Forest is substantially more accurate (+0.296 ROC-AUC, a 20× lower false-positive rate) but about 460× slower per row — a real, measured accuracy/latency tradeoff, not a one-sided win. The statistical baseline's recall is actually slightly higher, meaning it catches a few more true attacks at the cost of far more false alarms — a SOC tuning choice between "catch more, review more noise" and "catch slightly fewer, trust the alert." No tuning was done to make either number look better; this is the result as measured on 2026-09-12, with both input files' SHA-256 hashes recorded alongside the run for reproducibility.

This head-to-head is itself a deliverable: `sm_ml_training.benchmark` is a reusable harness (`python -m sm_ml_training benchmark --dataset nsl-kdd --train <path> --test <path>`) that stores dataset id, both file hashes, preprocessing version, split sizes, model parameters, random seed, and every metric — so the exact run can be reproduced or re-run against a different dataset later.

How this compares to traditional SIEM / rule-based IDS

Dimension	Traditional SIEM / signature IDS (e.g. Snort rules, basic Splunk correlation)	SentinelMesh
Detection method	Static signatures / threshold rules only	Rules + unsupervised ML anomaly detection (catches behavior that doesn't match a known signature)
Output unit	Thousands of discrete, unlinked alerts	Alerts automatically correlated into **attack chains** (`forming → active`) with a threat score
Threat-intel mapping	Usually manual or a separate paid feed	Built-in, imported **MITRE ATT&CK** catalog, automatically mapped to chains/detections

Dimension	Traditional SIEM / signature IDS (e.g. Snort rules, basic Splunk correlation)	SentinelMesh
Querying	Hand-written query language (SPL/KQL/Cypher)	**Natural-language hunting** — but compiled to a closed, validated `QueryPlan`, never a raw LLM-generated query (the usual way an "AI SOC" demo becomes a prompt-injection or data-exfil risk)
Explaining an incident	Analyst manually pivots across tools	LLM-grounded explanation, evidence-only prompting, explicitly labels evidence vs. inference vs. prediction
Testing detection coverage	Live production only (risky), or a separate disconnected test rig	Built-in **simulation + deception** engine: four synthetic attack templates (APT, ransomware, insider, brute-force) run in an isolated environment, visibly badged SIMULATION, never mixed with real data
Predicting what's next	Not typically offered	**Threat memory**: pgvector similarity search over past incidents + deterministic heuristic prediction, always labelled as a prediction with its basis, never presented as fact
Multi-tenancy	Often bolted on or absent in open-source tools	Tenant isolation enforced server-side at every layer (Identity → Tenant → Role → Permission → Resource), tested explicitly
Deployment	Varies widely in production-readiness	Helm chart for Kubernetes, HPA/PDB/NetworkPolicy per service, verified end-to-end on a real cluster
Claims about accuracy	Often marketing numbers with no reproducible methodology	Every number in this document is a real, reproducible, hash-pinned benchmark run — or explicitly marked `NOT VERIFIED`

The core architectural bet that differs from most "AI SOC" products on the market: **the LLM is never given standing privilege.** It cannot write to the graph, cannot execute an arbitrary query, and cannot auto-remediate — it only explains evidence it's handed and proposes a query plan that is validated before execution. This avoids the two failure modes most LLM-in-the-SOC-loop products run into: prompt-injected tool calls and hallucinated query results presented as fact.

What was achieved

- **19 phases (0–18) shipped and CI-verified** — architecture lock → foundation/auth → ingestion → event bus → graph → ML detection → threat intel/MITRE → correlation → graph serving → SOC frontend → AI analyst → NL hunting → simulation/deception/digital twin → threat memory/prediction → reporting → observability/benchmarking → Kubernetes → testing/security hardening → final integration + demo mode.
- **20 services + 5 shared packages**, one values-driven Helm chart (not 14 near-duplicate charts) covering every real service's Deployment/Service/HPA/PodDisruptionBudget/NetworkPolicy, plus self-hosted StatefulSets for every stateful dependency.
- **~200 automated test files** (182 Python `test_*.py`, 17 frontend `*.test.tsx`), 1,096 tests added in the Phase 17 hardening pass alone, run in CI against **real** PostgreSQL 16, Redis 7, Redpanda, and Neo4j 5 (not mocks) — CI fails outright if a dependency is unreachable rather than silently skipping.
- **A real trained, deployed Isolation Forest model** serving live anomaly scores, plus a reproducible, hash-pinned benchmark harness comparing it against a from-scratch statistical baseline (results above).
- **Real dataset replay through the real pipeline**: NSL-KDD rows ingested through the authenticated `POST /api/v1/ingest/batch` endpoint (not a DB shortcut), producing real detections, real attack-chain correlation, and real MITRE ATT&CK technique mappings (e.g. a brute-force scenario correctly mapped to T1110).
- **A real Kubernetes verification**: all 6 self-hosted stateful components and all 14 running app services reached genuine `1/1 Running` on a local `kind` cluster, with three real image/security-context bugs found and fixed by actually deploying (not just writing manifests) — one migration-Job connectivity issue honestly left unresolved and documented rather than hidden.
- **A working end-to-end demo path**: one-command `start.bat` / `stop.bat`, a seed script that provisions a tenant/admin/sensor with configurable credentials, a web-based dataset-upload page, a Detections list/detail UI, and a live SOC dashboard — all exercised against the real running stack, not screenshots.

- **37 of 38 architecture requirements traced** end-to-end (Requirement → Subsystem → Service → File → API/Event/Schema → Test → Verification), with the 2 known gaps named explicitly rather than silently dropped.
- **A non-fabrication discipline carried through 19 phases**: every performance, benchmark, or deployment claim in the project's own documentation is either a real measured number with its method recorded, or an explicit `NOT VERIFIED — REQUIRES <specific missing thing>` — verified at several points in this build by deliberately grepping the docs for fabricated-sounding claim language and confirming only negations/prohibitions matched.

Known limitations (stated honestly, not hidden)

An interviewer or professor who finds these by asking will trust the rest of the document far more than if they weren't here.

- **No deep-learning model is actually trained.** The autoencoder and GNN (GraphSAGE/GAT) have real, complete serving code and architecture specs, but zero trained checkpoints — calling them raises an explicit `ModelNotTrained` / `GraphModelNotTrained` rather than a fabricated score. The only trained, deployed detector is the classical Isolation Forest.
- **Only one labeled benchmark dataset (NSL-KDD) was actually usable.** CICIDS2017, UNSW-NB15, and LANL were investigated and could not be benchmarked — no labeled copy was available locally — so no number is claimed for any of them.
- **Final integration status is explicitly `PARTIAL`, not `COMPLETE`.** Two requirements (R18, R36) are missing from the traceability matrix with no recorded reason, inherited from an earlier session with no access to the primary source to fill them in.
- **Interactive OIDC/SSO browser login is `NOT VERIFIED`** in the local Docker Compose topology specifically — Keycloak can't serve two different self-identities to the browser and the container network without a reverse proxy this compose file doesn't have. The local email/password login path is fully verified and is what the demo uses.
- **One Kubernetes issue was not resolved**: a short-lived database-migration Job's Postgres connection repeatedly failed/timed out in the local `kind` cluster, while a long-lived service using the identical code path worked fine. Root cause was investigated but not conclusively isolated.
- **Flink (cluster stream processing) was deliberately not implemented** — the stream jobs today are stateless plain Python; this is a recorded architectural decision (ADR-010), not an oversight, and is fine at current scale.
- **No live LLM provider was ever called against the real Anthropic API in verification** — the HTTP boundary is real, tested code, but every demonstrated run used the deterministic, network-free adapter; a real API key would be required to verify live LLM

behavior.

- **Simulation/synthetic data is not a substitute for real attack traffic.** The four synthetic scenario templates are deterministic and useful for testing the pipeline end-to-end, but they are not adversarial, evolving, or representative of a real attacker adapting to defenses — they are explicitly badged SIMULATION everywhere in the UI so this distinction is never lost on a viewer.
- **CI does not stand up a real Kubernetes cluster.** The Phase 16 Kubernetes verification was a one-time local `kind` run, not something re-verified automatically on every push — a later code change could silently break it without CI catching it.

Future roadmap

Near-term (completes work already architected):

- **Train the autoencoder and GNN** against a real labeled dataset (or a graph-structured derivative of NSL-KDD/CICIDS2017) and run the same reproducible benchmark harness against them — the serving code is ready; this is purely a data/compute/training-run task, and would let a true deep-learning-vs-classical-ML comparison be published with real numbers instead of an architecture description.
- **Acquire and benchmark CICIDS2017 and UNSW-NB15** (both modern, more diverse datasets than NSL-KDD, which is ultimately a cleaned 1999-era dataset) to validate detection generalizes beyond one benchmark's traffic patterns.
- **Resolve the Kubernetes migration-Job connectivity issue** and re-verify the full Helm deployment, then wire a `kind`-based job into CI so the Kubernetes path is checked on every push, not just once.
- **Fill the two missing requirement-traceability entries (R18, R36)** against the primary architecture source.

Medium-term (new capability):

- **Active-learning feedback loop**: let an analyst's "confirmed" / "false positive" triage decisions on detections feed back into periodic Isolation Forest / autoencoder retraining, closing the loop between human judgment and the model instead of a static, one-time-trained artifact.
- **SOAR-style automated response** (currently out of scope by design — the LLM is deliberately never given standing privilege): a tightly-scoped, human-approved remediation action framework, keeping the same "LLM proposes, never executes unchecked" boundary.
- **A sequence/temporal deep-learning model** (e.g. an LSTM or small Transformer over the event timeline already captured by `sm_ml.temporal`) to detect slow, multi-stage attacks that a single-row anomaly score structurally can't see — the temporal event-

stitching infrastructure already exists and currently feeds heuristic, not learned, progression detection.
- **Multi-provider LLM support beyond the Anthropic-shaped boundary** (OpenAI, a self-hosted local model) — the provider-adapter interface was deliberately designed to make this an additional adapter, not a rearchitecture.
- **Federated / cross-tenant threat intelligence** building on the existing `federation-service` (currently a scaffolded placeholder, not yet implemented) — sharing anonymized attack patterns across tenants without sharing raw data.

Longer-term (research-shaped):

- **Self-supervised pretraining on unlabeled traffic** before fine-tuning the anomaly head — most real-world telemetry is unlabeled, and the project already treats "no labels" as the normal case (both the statistical baseline and Isolation Forest are unsupervised), so this is a natural extension rather than a new assumption.
- **Explainability for the ML scores themselves** (e.g. SHAP values for Isolation Forest, attention-weight inspection for GAT) surfaced in the AI analyst's grounded explanation, so "why did the model flag this" has a model-level answer, not only a rule-level one.

Interview & viva cheat-sheet

"Does this project use deep learning?" — The production, trained, deployed detector is classical ML (Isolation Forest), not deep learning. Two deep-learning architectures (an autoencoder and a GraphSAGE/GAT graph neural network, both PyTorch) are fully designed with real serving/inference code, but have no trained checkpoint — say this directly, then pivot to *why* (see below). Never claim a trained deep model that doesn't exist.

"Why Isolation Forest and not a neural network?" — Unsupervised, needs no labeled attacks, cheap to train, robust on tabular features, and interpretable enough to benchmark honestly in the time available. Deep models need more data/compute and were architected for a later phase rather than rushed with a fabricated accuracy number.

"What's your model's accuracy?" — Give the real numbers: ROC-AUC 0.935, precision 0.961, recall 0.621, F1 0.755, FPR 0.033, on NSL-KDD's test split, measured 2026-09-12, reproducible via a hash-pinned benchmark harness. Always name the dataset — never state accuracy without it.

"Why NSL-KDD?" — Cleaned/de-duplicated successor to KDD'99, the most widely published IDS benchmark (comparable to prior literature); the three alternatives considered (CICIDS2017, UNSW-NB15, LANL) weren't available locally in usable labeled form, so no number was fabricated for them.

"How does the LLM fit in, and isn't that risky?" — The LLM only does two things: explain an already-detected, already-evidenced chain, and translate a natural-language question into a closed `QueryPlan` schema that's validated before execution — it never generates or executes a raw query, never gets standing privilege, and never auto-remediates. This directly avoids prompt-injection and hallucinated-query risk, the two most common failure modes in "AI SOC" products.

"What's the system architecture?" — 20 microservices in a strict pipeline (ingest → normalize → stream → graph → detect → correlate → explain), Kafka as the durable event bus, Neo4j as the single-writer attack graph, Postgres for the control plane + pgvector, Redis for cache/session only (never the bus), deployed via one values-driven Helm chart.

"How is this better than Splunk / a SIEM?" — It's not a replacement for log aggregation at that scale; it's a complementary detection + correlation + explanation layer: automatic attack-chain reconstruction, built-in MITRE mapping, safe natural-language hunting, and a closed-loop simulation/deception testbed — capabilities a raw SIEM doesn't provide out of the box.

"What would you do with more time?" — Train the autoencoder/GNN and publish a real deep-vs-classical comparison, acquire CICIDS2017/UNSW-NB15, close the Kubernetes migration-Job bug, and add an active-learning loop so analyst triage decisions retrain the model.

"What's the weakest part of this project?" — Answer honestly from the Known Limitations section above — the untrained deep models and the single-dataset benchmark are the two most defensible to name first; naming your own weakest point unprompted reads as more credible than waiting to be caught out.

"Is any of this fabricated or exaggerated?" — No — the project was built under an explicit "never fabricate a benchmark/accuracy/deployment claim" rule enforced through all 19 phases, re-verified by grepping the documentation for performance-claim language and confirming only negations/prohibitions matched, not actual unverified numbers.

Post-remediation update (2026-10-06)

Everything above this line was written before a dedicated remediation pass. That pass changed the honest answer to "is the autoencoder/GNN trained?" from **no** to **yes** — read this section as the current state; where it conflicts with the AI/ML section above, this section wins.

Autoencoder and GraphSAGE/GAT are now actually trained, with real checkpoints, not just architecture. Real, measured benchmark on NSL-KDD (same methodology as before — benign-only fit, held-out test split, reused metric code):

Model	ROC-AUC	Note
Isolation Forest	0.935	Still the best performer
Statistical baseline	0.639	Unchanged
Autoencoder (125-dim one-hot)	0.434	**Worse than chance-adjacent** — real, reported as measured, not hidden
GraphSAGE / GAT	≈ 0.49	**Near-random** — root-caused, not hand-waved (next paragraph)

Why the GNN result is near-random, explained, not excused: the graph built from NSL-KDD (`scripts/build_graph_dataset_nsl_kdd.py`) has to invent placeholder IP structure because NSL-KDD carries none — every source node ends up with identical degree-1 structure regardless of whether it was an attack. `sm_ml.graph.construct` 's node features are **purely structural** (degree, clustering, timing) by design, for the platform's real multi-hop live attack graph — on this particular synthetic derivative, that design choice means the GNN literally cannot see the duration/bytes/protocol signal the tabular models see directly. This is the single most interview-useful finding in the whole remediation: a real, working, correctly-engineered training pipeline can still produce a bad model when the *data* doesn't carry the signal the *architecture* needs — a point many engineers only learn the hard way.

Kubernetes: four real bugs found and fixed, not one:

1. The migration Job's Helm hook annotation (`pre-upgrade`) meant it was **never created at all** on a first `helm install` — fixed with a `.Release.IsUpgrade` guard.
2. `socket.create_connection(timeout=2)` doesn't bound DNS lookup time, only the connect phase — a hung lookup could block the retry loop forever; fixed with `signal.alarm` .
3. The chart's own defaults set `SM_ENV=production` *and* `SM_KAFKA_SECURITY_PROTOCOL=PLAINTEXT` together — self-contradictory, tripping the app's own production safety guard on every single deploy. Fixed by defaulting to `staging` .
4. `--set <service>.replicas=1` silently did nothing — the real values path is `services.<name>.replicas` , one level deeper than it looked. This alone caused two different "it's just DNS flakiness under memory pressure" false conclusions before the real cause was found by actually reading `kubectl describe deploy` output instead of assuming.

After all four fixes: a real `helm install` on a real `kind` cluster produces a migration Job that reaches genuine `Complete`, and `api-gateway` rolls out successfully — reproduced both locally and on a real GitHub Actions runner.

CI: added a `kind-deploy` job that actually stands up a Kubernetes cluster on every push (the README used to say plainly that CI never did this). Took 5 real runs and 2 more real bugs (a test suite assumption + an environment-dependent `mypy` quirk) before it went green — that run history is itself evidence of real iteration, not a first-try success story.

Still genuinely blocked, not fabricated past: CICIDS2017 and UNSW-NB15 both now require manual registration at the source (checked live — no anonymous download exists any more); R18 and R36 in the requirements traceability matrix remain unrecoverable because the source document that defines them doesn't exist anywhere accessible, confirmed directly with the project owner. Full day-by-day account: `docs/PRIORITY_REMEDIATION_REPORT.md` in the repo.

Novelty, prior art, and why this project

Does anything like this already exist? Yes — be upfront about that. No individual capability here is unprecedented. What's being claimed as novel is a specific *combination*, and the engineering discipline behind it — not an invented algorithm.

What already exists, honestly mapped

Category	Real examples	What they have that SentinelMesh doesn't (yet)	What SentinelMesh has that they typically don't
Open-source SIEM / XDR	Wazuh, Security Onion, OSSEC, Elastic Security, TheHive+Cortex	Years of production hardening, huge detection-rule libraries, real deployments at scale	Native attack-graph reconstruction (not just correlated alerts), a constrained-not-raw LLM hunting interface, built-in simulation+deception in the same platform

Category	Real examples	What they have that SentinelMesh doesn't (yet)	What SentinelMesh has that they typically don't
Commercial AI-driven SOC	Microsoft Sentinel (Fusion ML, UEBA), CrowdStrike Falcon (threat graph), Darktrace (self-learning AI)	Real production ML at massive scale, enterprise support, years of threat-intel data	Fully open, inspectable source; an explicit non-fabrication evidence trail (published negative results, not just marketing accuracy numbers); zero standing LLM privilege by architecture, not by policy
Academic graph-ML for intrusion detection	Published GraphSAGE/GAT-on-NSL-KDD and CICIDS2017 papers exist	Rigorous statistical evaluation, peer review, often better-curated graph structure	Integration into a full running platform (ingestion → detection → correlation → explanation → reporting), not just a model trained in a notebook
Adversary emulation / simulation	MITRE CALDERA, Atomic Red Team	Mature, widely-used attack technique libraries	Simulation is wired directly into the same detection pipeline it tests, with automatic MITRE ATT&CK mapping of what the simulation triggered
"AI SOC analyst" products (2024–2026 startups)	Various commercial entrants	Live LLM deployments, real customer feedback loops	A specific, auditable safety pattern: the LLM never generates executable queries, only a closed `QueryPlan` schema that's validated before anything runs — many competitors in this exact space *do* let an LLM get closer to raw query generation

Why I built it

To go through the entire lifecycle a real cybersecurity-AI product actually requires — auth and multi-tenancy, event-driven microservices, a graph database, classical ML, deep learning, LLM integration with real safety constraints, Kubernetes, CI/CD — end to end, under a self-imposed rule that nothing gets claimed as working unless it was actually run and measured. That rule is the real point of the project as a learning exercise: it's easy to build something that *looks* finished; it's much harder to build something where every claim in the documentation has a command and a result behind it, including the claims that turned out badly (the GNN's near-random accuracy is in the report, not hidden).

How it's better than its predecessors — and where it genuinely isn't

Genuinely better / different:

- **Safety-first LLM integration.** Natural-language hunting compiles to a closed, validated `QueryPlan`, never raw Cypher — this specifically avoids the prompt-injection and hallucinated-query failure modes that are common, documented problems in LLM-assisted security tooling generally.
- **Transparency as a design principle, not an afterthought.** Every benchmark number in this project has its measurement method next to it; a result that came out badly (GNN ≈ random) is published with its root cause, not quietly dropped. Most commercial security-AI marketing does not do this.
- **One platform, not a stitched-together toolchain.** Ingestion, graph correlation, ML detection, LLM explanation, simulation/deception, and reporting share one event model and one tenant-isolation boundary, instead of being separate tools glued together with scripts.

Not better — said plainly, because overclaiming this would be exactly the kind of fabrication the project's own rules forbid:

- It does not outperform CrowdStrike, Darktrace, or Microsoft Sentinel at actual threat detection — those are mature products with years of real adversary data this project has never seen.
- The GNN result on NSL-KDD is genuinely worse than a classical method here — the project does not claim deep learning is better in general, only reports what was actually measured on this dataset.
- It has not been through a real security audit, red-team engagement, or production incident — everything verified here is local/CI verification, not battlefield-tested.

The honest one-line pitch for an interviewer: *"I didn't invent a new detection algorithm — I built the full, correctly-engineered pipeline a real security-AI product needs, including the parts most student projects skip (safety-constrained LLM use, Kubernetes*

verification, CI that actually deploys), and I can show you the exact commands and results behind every claim I make about it, including the ones that didn't work."